NANDHA ENGINEERING COLLEGE
ERODE - 638052

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT
OF
COMPUTER SCIENCE AND ENGINEERING

CANDY CRUSH

(22CSC07 – JAVA PROGRAMMING)

A PROJECT BASED LEARNING REPORT

Submitted by

DHARUN RAJ R (22CS021)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

DECEMBER 2023

NANDHA ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this report titled **" CANDY CRUSH "** is the bonafide work of **DHARUN RAJ R (22CS021)** who carried out the work under my supervision. Further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE OF HOD

Dr. D.Vanathi M.E.,Ph.D.,
Professor &
Head Department of CSE,
Nandha Engineering College,
Erode - 638052

SIGNATURE OF SUPERVISOR

Mrs. C.NAVAMANI M.E.,
Assistant Professor
Department of CSE,
Nandha Engineering College,
Erode - 638052

Submitted for End Semester Project Viva-Voce Examination held on……

INDEX

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	**INTRODUCTION**	5
2	**HOW TO PLAY**	6
3	**LITERATURE SURVEY**	
4	**REQUIREMENTS**	8
	4.1 HARDWARE REQUIREMENTS	
	4.2 SOFTWARE REQUIREMENTS	
5	**DESIGN**	9
	5.1 EXISTING SYSTEM	
	5.2 PROPOSED SYSTEM	
	5.3 SYSTEM DESIGN	
6	**FLOWCHART**	11
7	**IMPLEMENTATION**	12
8	**RESULTS & SNAPSHOTS**	15
9	**MODULE**	21
10	**APPENDIX**	24
11	**CONCLUSION AND FUTURE SCOPE**	43
	11.1 CONCLUSION	
	11.2 FUTURE SCOPE	44
12	**REFERENCES**	47

ABSTRACT

This Java-based Candy Crush Mini Project showcases the intrinsic capability of graphics to provide an intuitive and engaging interface for interacting with a computer. Harnessing our well-developed visual pattern recognition, the game swiftly processes and presents pictorial data. Drawing parallels with the historical significance of photography and television, interactive computer graphics emerges as a paramount means of producing dynamic and interactive images. Beyond merely depicting real-world candy objects, this project highlights the versatility of the computer in creating visuals of both tangible confections and abstract, synthetic elements integral to the Candy Crush gameplay. By extending beyond traditional boundaries, the game embodies the fusion of programming and graphical representation, offering players an immersive experience that spans from the delightful imagery of candies to the dynamic abstraction of game mechanics.

1. INTRODUCTION

The Candy Crush game project is a Java Swing application designed to replicate the fundamental mechanics of the popular Candy Crush puzzle game. Its purpose is to provide users with an entertaining and interactive experience where they can match candies of the same color to score points within a limited number of moves.

1.1Purpose and Functionality:

The project aims to offer a simplified version of the Candy Crush game, allowing players to enjoy the classic matching gameplay. Users can swap adjacent candies either horizontally or vertically to create matches, with the goal of achieving the highest score possible. The game includes features such as a limited number of moves, scoring based on the length of candy matches, and dynamic generation of new candies after matches are made**.**

1.2Problem or Need Addressed:

The project addresses the need for a casual and enjoyable gaming experience. It caters to individuals looking for a straightforward implementation of the Candy Crush concept, providing a platform for entertainment and relaxation.

How to Play

To play Candy Crush, the user must:

1. Click on a candy to select it.
2. Click on an adjacent candy to swap positions.
3. Create matches of three or more candies of the same color horizontally or vertically.
4. Score points based on the length of candy matches.
5. Use a limited number of moves strategically to achieve the highest score.
6. Limited number of moves available.
7. Moves decrease with each player action.

LITERATURE SURVEY

+ The provided Java code represents a basic implementation of the popular game Candy Crush using Java Swing. This code demonstrates fundamental concepts in graphical user interface (GUI) programming with Java, focusing on event handling, button interaction, and dynamic content updates. The game structure includes a grid of buttons, each representing a candy, and employs a simple matching algorithm to detect and process candy matches, updating the player's score accordingly. While the code serves as a practical introduction to Java Swing for educational purposes, it is worth noting that Java Swing is an older GUI framework, and more modern alternatives or game development platforms may be preferable for complex and performance-sensitive applications.

The literature survey for this code would likely involve exploring introductory Java Swing tutorials and documentation to understand the basics of GUI programming in Java. Additionally, it could include an overview of foundational game development concepts, such as event-driven programming, graphical rendering, and basic game mechanics. However, for more advanced game development, researchers and developers may refer to literature on modern game development frameworks, engines, and design patterns that offer enhanced capabilities and support for creating interactive and engaging gaming experiences.

SYSTEM REQUIREMENTS SPECIFICATION

3.1 HARDWARE REQUIREMENTS

Minimum hardware specification

- Microprocessor: **1.0 GHz** and above CPU based on either AMD or INTEL Microprocessor Architecture
- Main memory : **512 MB RAM**
- Hard Disk : **40 GB**
- Keyboard: **QWERTY** Keyboard
- Mouse :**2 or 3** Button mouse
- Monitor : **1024 x 768** display resolution

3.2 SOFTWARE REQUIREMENTS

Minimum software specification

- Operating system : WINDOWS 8
- Tool Used : IntalIJ
- OPENGL Library
- X86
- X64(WoW)
- Mouse Driver
- Graphics Driver
- Java JDK 17

DESIGN

4.1 **Existing System:**

The existing system represents the current state of the Candy Crush game code. In this implementation, the game leverages Java Swing for creating the graphical user interface and incorporates a grid of buttons representing different types of candies. The game logic involves swapping candies, checking for matches, updating the score, and managing the limited number of moves. The graphical elements are enhanced with candy images, and the game provides an end-of-game dialog displaying the final score and offering the option to play again or exit. The existing system serves as a foundation for the proposed system but may have opportunities for improvement in terms of features, scalability, or code organization.

4.2 Proposed System:

The proposed system suggests potential enhancements and improvements to the existing Candy Crush game. Possible areas for enhancement include:

User Interface Improvements: Enhance the graphical user interface for a more modern and visually appealing design.

Additional Features: Introduce new features, levels, or game elements to make the gameplay more engaging and challenging.

Scalability: Design the system to support future expansions, such as adding more candy types or levels.

Code Refactoring: Evaluate the code structure for readability, maintainability, and adherence to best practices.

The proposed system aims to build upon the strengths of the existing system and address any identified limitations to create a more robust and enjoyable gaming experience.

4.3 System Design:

The system design encompasses the architectural and structural aspects of the Candy Crush game code. Key design considerations include:

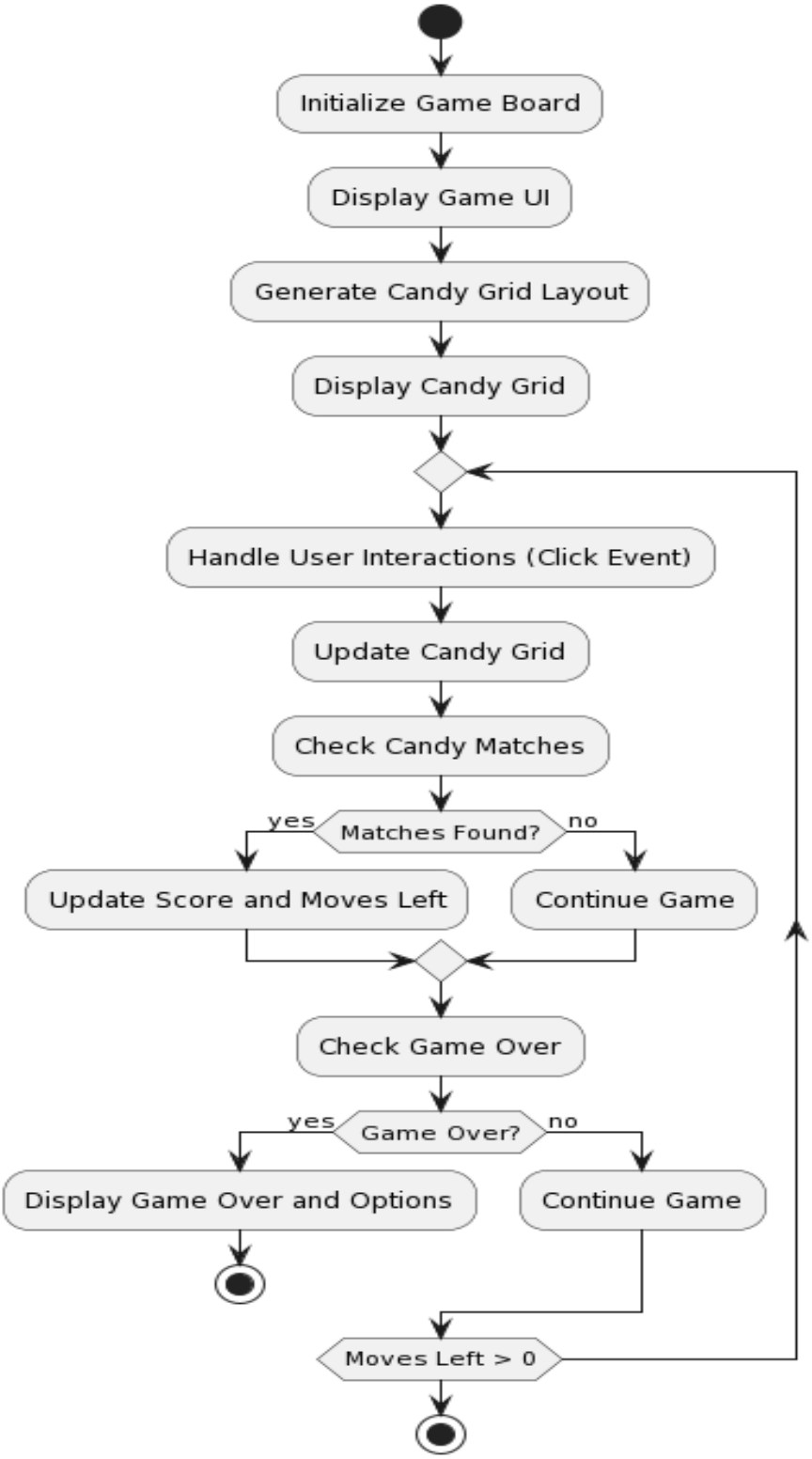
Class Hierarchy: Review and refine the class structure to ensure a well-organized and modular codebase.

Code Modularity: Emphasize modular design principles to enhance code readability and maintainability.

Scalability: Design the system to accommodate potential future enhancements, such as additional features or levels.

User Interface Design: Consider improvements to the graphical user interface, including layout, aesthetics, and user interaction.

FLOWCHART


Initialize Game Board
Display Game UI
Generate Candy Grid Layout
Display Candy Grid
Handle User Interactions (Click Event)
Update Candy Grid
Check Candy Matches
yes
Matches Found?
no
Update Score and Moves Left
Continue Game
Check Game Over
yes
Game Over?
no
Display Game Over and Options
Continue Game
Moves Left > 0

IMPLEMENTATION

1. Introduction:

The Java code under consideration orchestrates the development of a simplified Candy Crush game, leveraging the Swing library to establish an intuitive graphical user interface. The Candy class not only encapsulates the game's core logic but also meticulously structures the various graphical elements, rendering an engaging and visually pleasing gaming experience.

2. Class Structure and Initialization:

Within the Candy class, arrays are employed to store the colors, icons, and URLs pertaining to distinct candy types. Initialization of these arrays sets the stage for the subsequent creation of the game's visual elements. The game grid, represented as a 6x10 matrix of JButtons enclosed within JPanels, is the canvas upon which the candy-matching dynamics unfold. Each JButton embodies a candy with its unique color, icon, and action command, establishing a coherent and interactive game grid.

The JFrame (jf) and other graphical components are initialized to craft the overarching layout of the game. The constructor additionally facilitates the loading of candy images from resource URLs and configures the initial state of the game, establishing a foundation for subsequent user interactions.

3. ActionListener Implementation:

The Action class, nested within Candy, implements the ActionListener interface, a pivotal aspect of facilitating user interaction. The actionPerformed method serves as the linchpin for handling user clicks on the candy buttons. A meticulous process is employed to discern the nature of the user's interaction, determining whether it constitutes the first or second candy in a potential swap. This method orchestrates the visual swap of candies and triggers the consequential cascade of checks for matching candies. The concurrent update of the score and moves counters enriches

the gaming experience, and the potential conclusion of the game is managed when no moves remain.

4. Matching and Replacement Logic:

The crux of the game lies in the check method, an algorithmic tour de force that systematically scrutinizes the game grid for horizontal and vertical candy matches. When a cluster of three or more matching candies is identified, these candies undergo replacement with fresh ones, while the player's score increments. The recursive nature of this method ensures that cascading matches are not overlooked, fostering a dynamic and continuous gameplay experience.

5. Animation and Delay:

Introducing a cinematic touch, the sleep method injects a calculated delay into the game, creating a brief yet visually impactful pause after candy matches and before replacements. This temporal nuance contributes significantly to the player's immersion, allowing them to absorb the outcome of their moves. The down method plays a complementary role by orchestrating the descent of candies after matches, underscoring the quintessential cascading effect that characterizes Candy Crush.

6. Random Candy Generation:

The element of unpredictability and variety in the game is orchestrated by the rand method. This function generates a random number within a defined range, shaping the type of candy instantiated during grid initialization or replacement after matches. This stochasticity injects an inherent replayability factor, ensuring that each gaming session unfolds with a unique arrangement of candies.

7. Main Method and Game Initialization:

The main method emerges as the point of ingress for the entire program, initiating the game by creating an instance of the Candy class. In this phase, the graphical user interface is instantiated and unveiled, providing a tangible platform for players to engage with the candy buttons and formulate strategic moves.

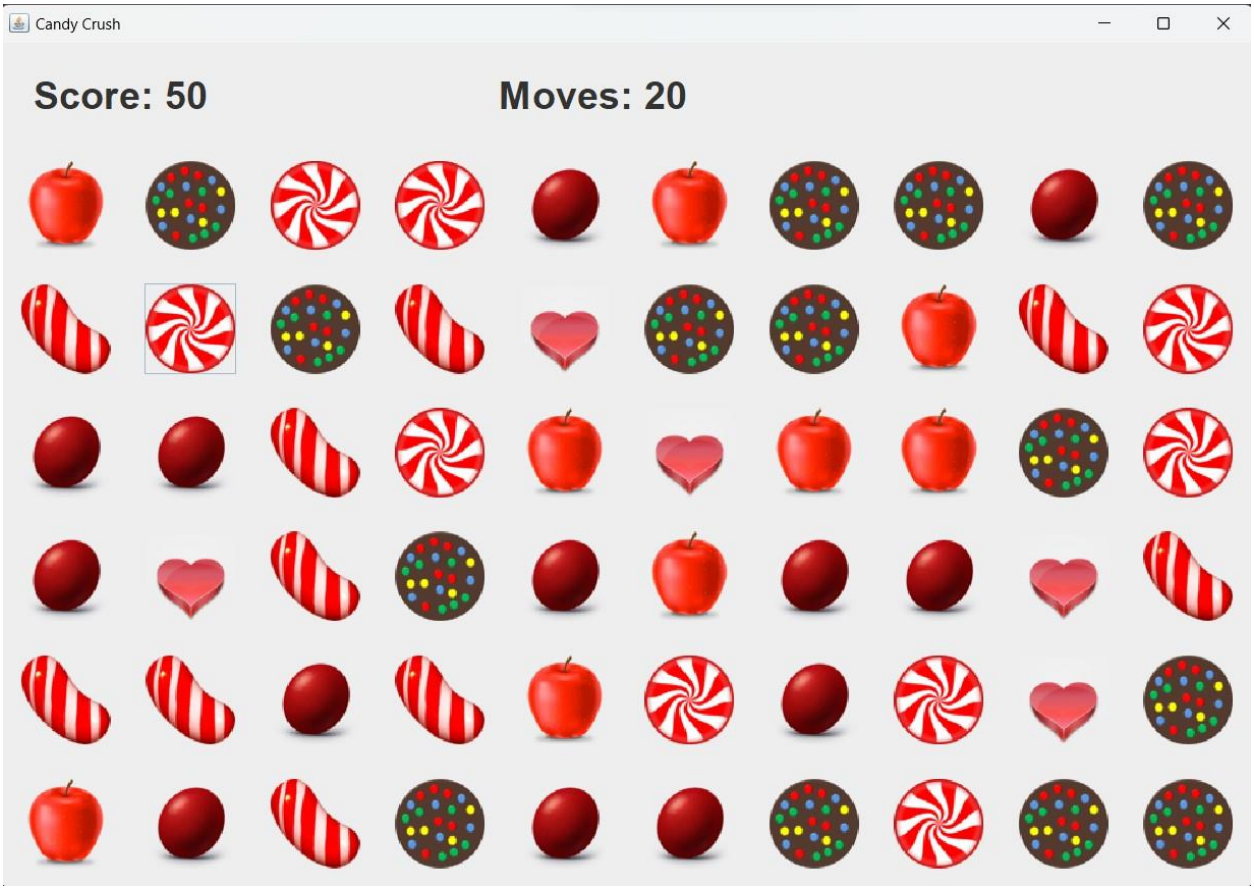
8. Game Over Handling:

The code incorporates an astute game over mechanism, triggered when the player exhausts all available moves. A JOptionPane dialog elegantly surfaces, displaying the player's final score and extending the choice to either embark on another round or gracefully exit the game. This decisive juncture exemplifies the code's ability to manage the game lifecycle and user interaction seamlessly.

9. Graphical User Interface (GUI) Components:

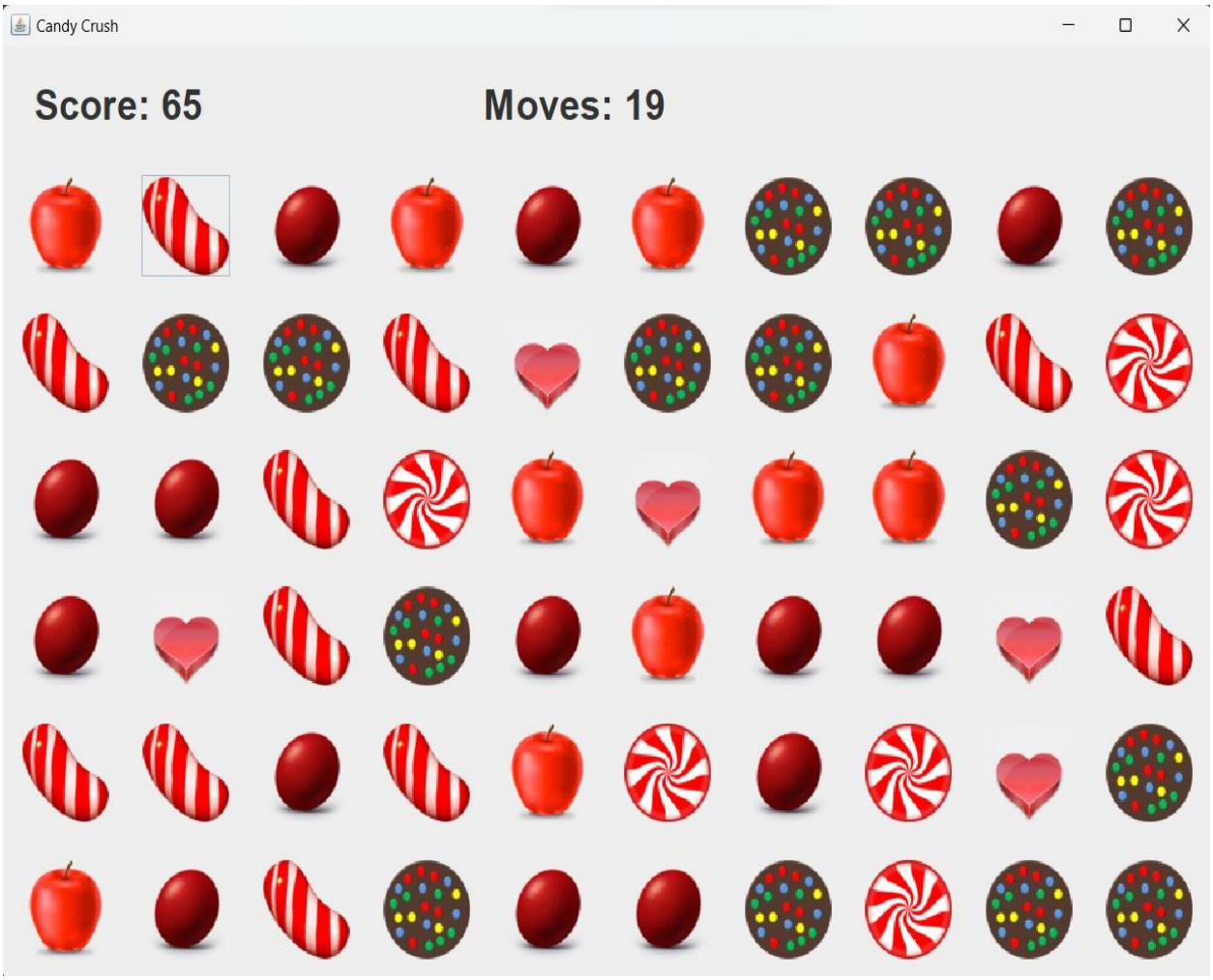
At the heart of the Candy Crush game lies an elaborate graphical user interface. The JFrame orchestrates the overarching layout, complemented by JPanels, JLabels, and the bedrock of interactivity – JButtons representing individual candies. ImageIcon proves instrumental in imbuing these candies with visual identities, rendering them recognizable and distinct. The meticulous arrangement of these components collectively fashions an immersive and aesthetically pleasing game environment.

RESULTS & SNAPSHOTS

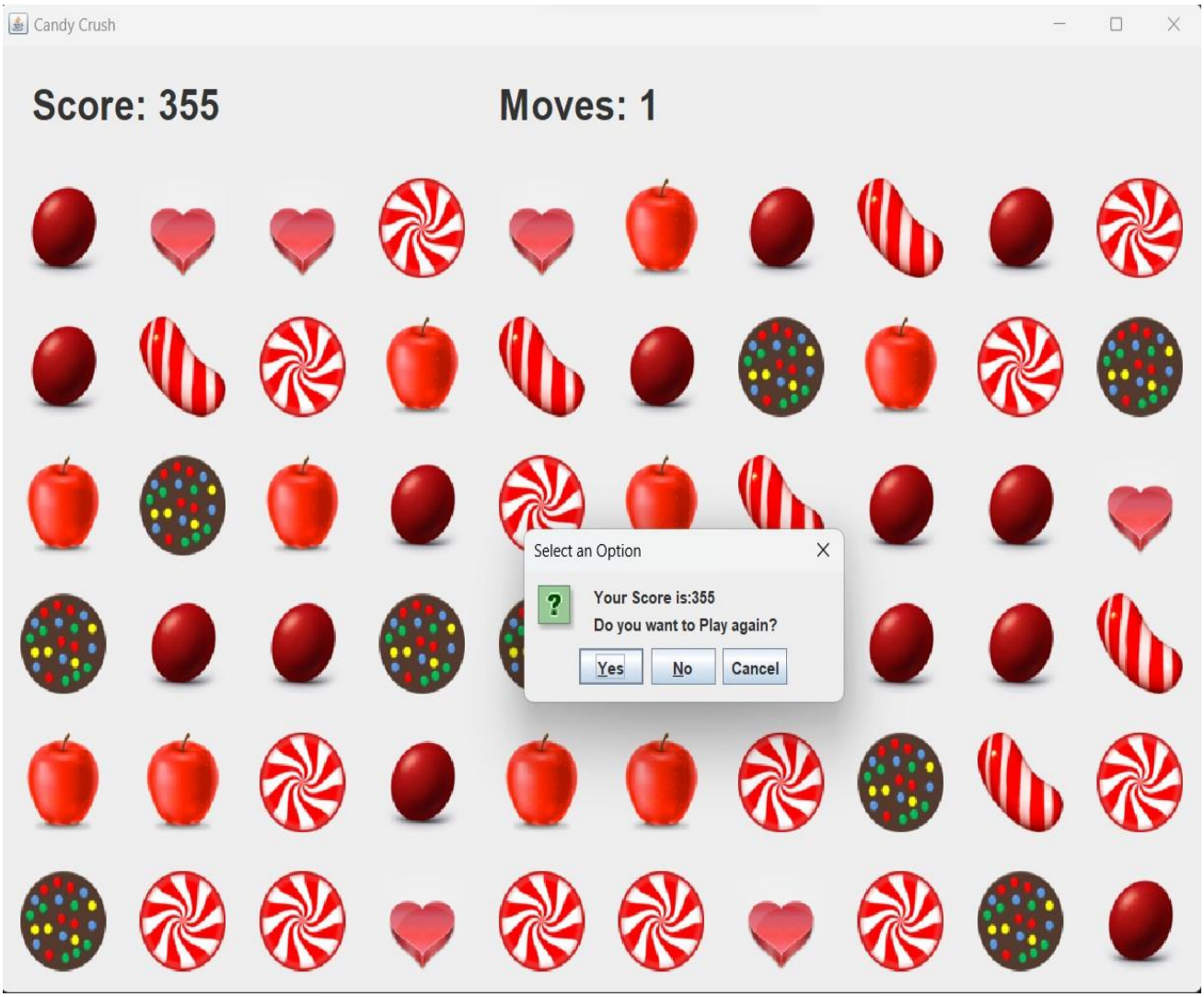


Fg :1

After running the code the game start's and ready to play.The score be 50 and the moves limits is 20 at being.

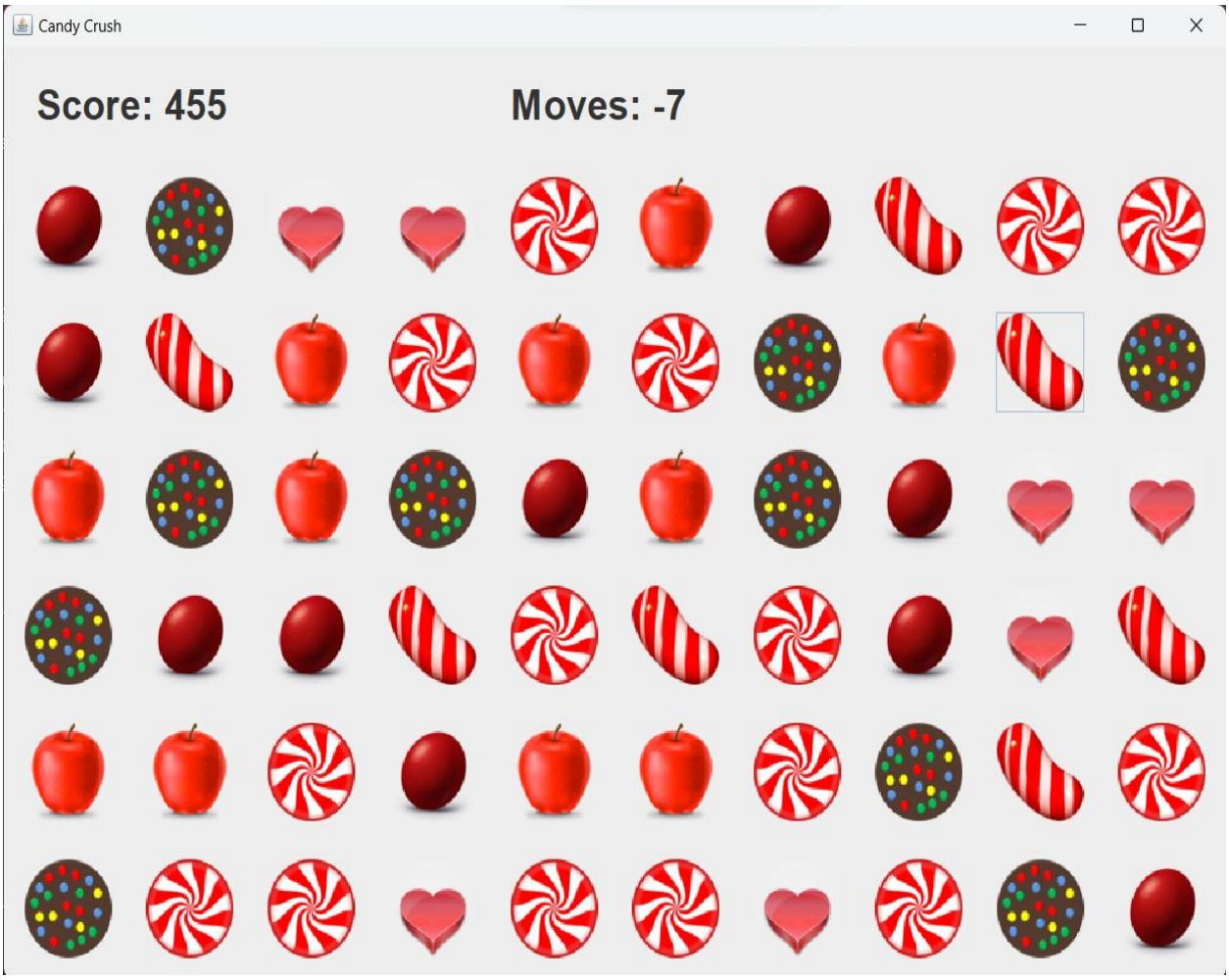


Fg: 2
After making a move. Score update and number of move decrease's



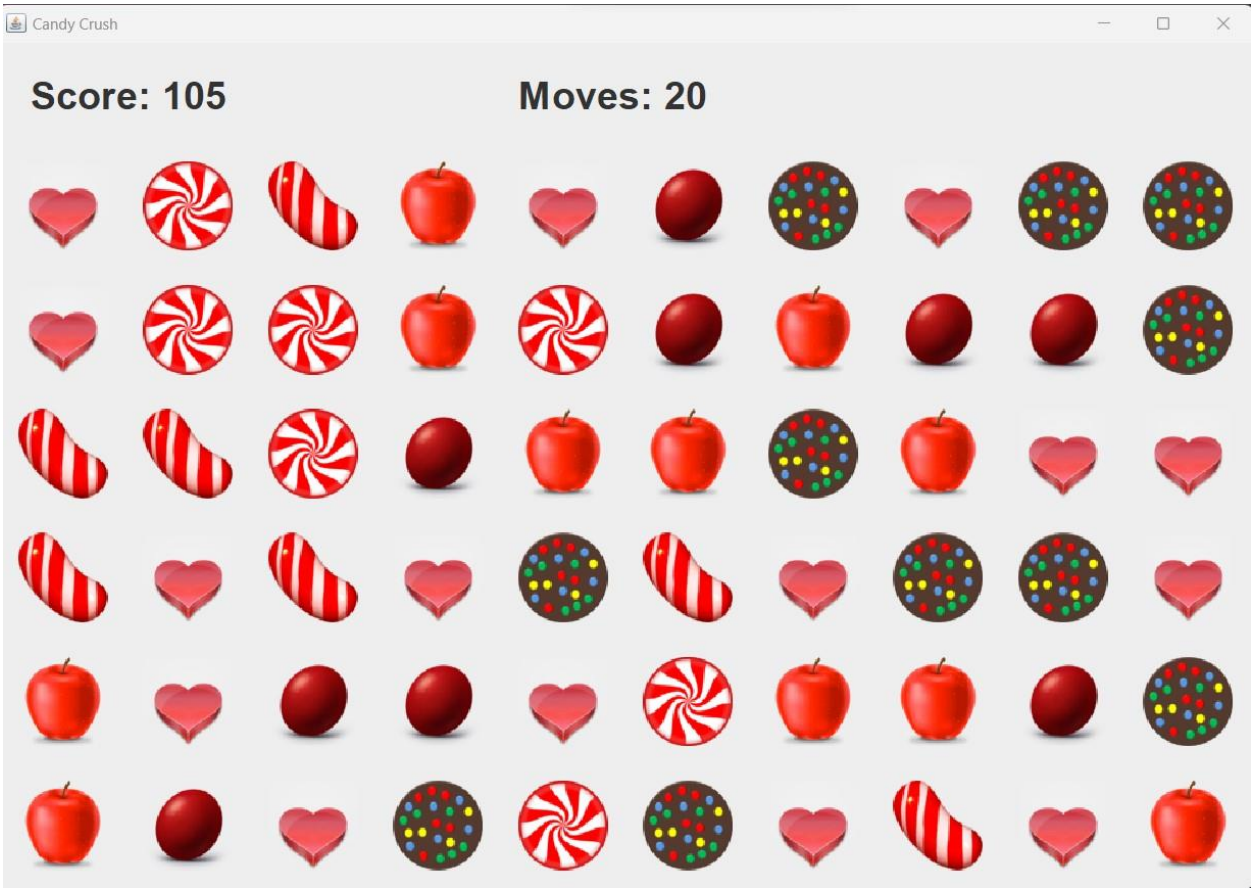
Fg; 3

After the move limits reached 1 the game end and the dialog box display's as show in above Fg:3

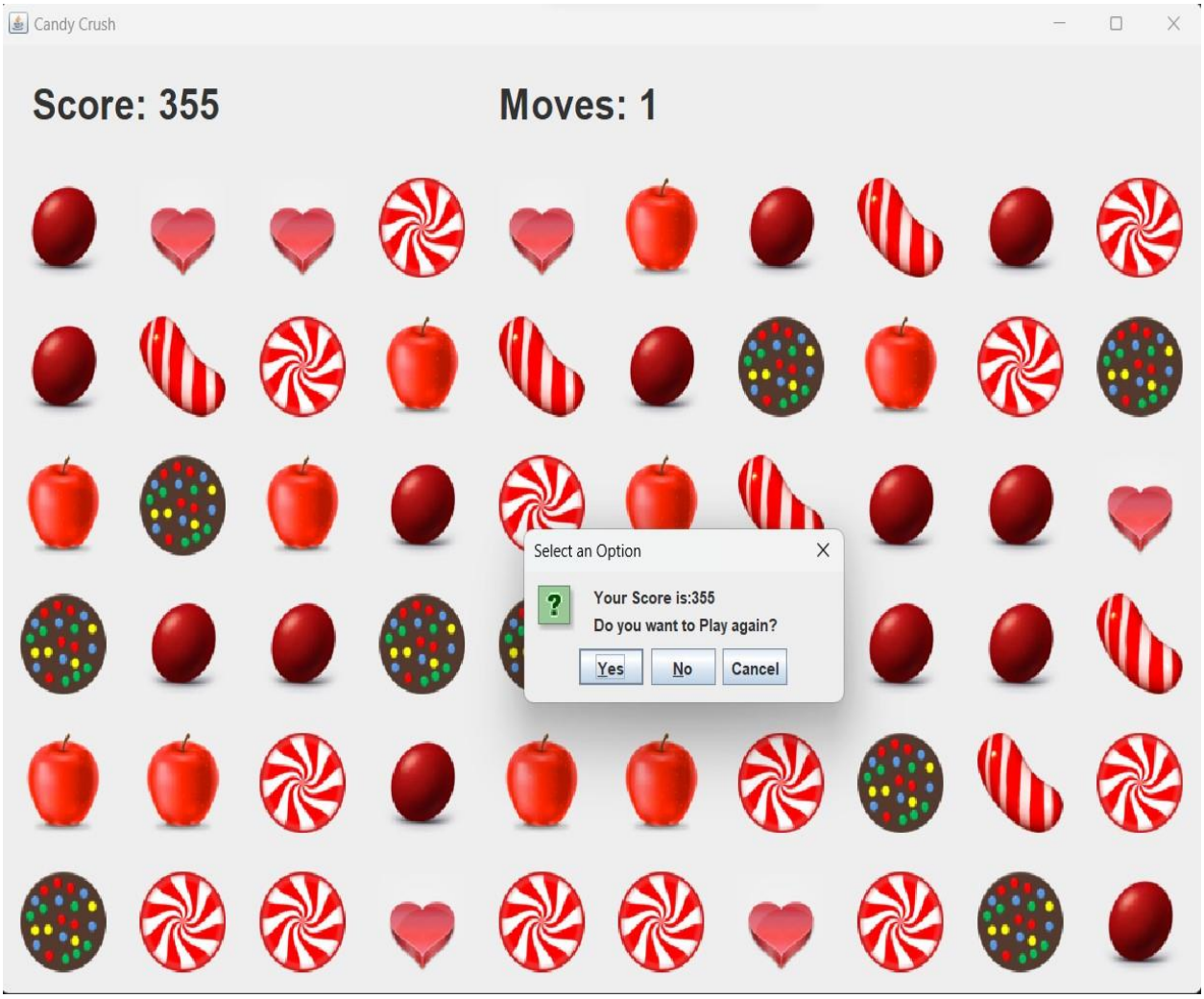


Fg: 4

After clicking Cancle button then the move's go in negative and score updates and game continues



The game restare’s and player can paly farther.



If we press the No button then it exit the game.

MODULES

Let's discuss the imported modules and their functions:

1. javax.swing:

Function: This package provides a set of GUI components for building graphical user interfaces in Java. In the Candy Crush code, it is used for creating windows (JFrame), panels (JPanel), buttons (JButton), labels (JLabel), and text fields (JTextField), forming the visual elements of the game.

2. java.awt:

Function: The Abstract Window Toolkit (AWT) is a package that provides essentials for creating graphical user interfaces, such as layout managers and dimensions. In the Candy Crush code, it is used for setting the layout of the panels and defining the dimensions of certain GUI elements.

3. java.awt.event.ActionEvent:

Function: This class represents an action event that can be generated when a user interacts with a graphical component, like clicking a button. In the Candy Crush code, it is used in conjunction with the ActionListener interface to handle button clicks within the game grid.

4. java.awt.event.ActionListener:

Function: This interface is used for receiving action events, typically from GUI components like buttons. In the Candy Crush code, the Action class implements this interface to define the actions to be taken when candy buttons are clicked.

5. java.net.URL:

Function: The URL class represents a Uniform Resource Locator, which is used to specify the location of resources like images. In the Candy Crush code, it is utilized to store the URLs of different candy images that are loaded into the game.

6. java.util.Random:

Function: The Random class provides methods for generating pseudorandom numbers. In the Candy Crush code, it is used to generate random numbers for selecting different types of candies, ensuring a varied and unpredictable initial configuration of candies on the game grid.

7. javax.swing.ImageIcon:

Function: This class is used to create and manage image icons, which can be displayed on GUI components like buttons. In the Candy Crush code, it is used to set images for the different types of candies displayed on the buttons.

8. javax.swing.JOptionPane:

Function: The JOptionPane class provides methods to create standard dialog boxes, such as message dialogs, input dialogs, and confirm dialogs. In the Candy Crush code, it is used to display a confirmation dialog at the end of the game, showing the player's score and prompting whether to play again or exit.

9. java.util.Arrays:

Function: The Arrays class provides various methods for manipulating arrays in Java. In the Candy Crush code, arrays are used to store colors, image icons, and URLs associated with different types of candies.

10. java.lang.Thread:

Function: The Thread class is part of Java's multithreading support. In the Candy Crush code, it is used in the sleep method to introduce delays, possibly for visual effects or timing certain actions within the game.

APPENDIX

SOURCE CODE

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.net.URL;
import java.util.Random;

/**
 * Created by Muhammad Soban on 1/27/2015.
 */
public class Candy {

    Color []cl =  new Color[6];
    ImageIcon []icon = new ImageIcon[6];
    URL []url = new URL[6];
```

```
    int x=-1,y=-1,score=0,moves=20;
    JFrame jf = new JFrame();
    JPanel jp = new JPanel(new GridLayout(6,10));
    JPanel jp1 = new JPanel(new BorderLayout());
    JButton [][] bn = new JButton[6][10];
    JTextField jt = new JTextField();
    JLabel jb = new JLabel("   Score: "+score+"
Moves: "+moves);

    public Candy()
    {
        //JLabel pic = new JLabel(new
ImageIcon(getClass().getResource("/full.jpg")));
        //pic.setLayout(new BorderLayout());
        //jf.setContentPane(pic);

        Dimension d1 = new Dimension();
        d1.height = 80;
        jb.setPreferredSize(d1);
        jb.setFont(jb.getFont().deriveFont(30.0f));

        Container cn = jf.getContentPane();
```

```
        cn.setLayout(new BorderLayout());
        cn.add(jp1, BorderLayout.EAST);
        cn.add(jb,BorderLayout.NORTH);
        cn.add(jp);

        //pic.add(jp);

        jf.setTitle("Candy Crush");
        jf.setSize(1000,700);
        jf.setLocation(200,0);
        //jf.setUndecorated(true);

jf.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLO
SE);




        url[0] = getClass().getResource("/candy70x70.png");
        url[1] = getClass().getResource("/chocolate70x70.png");
        url[2] = getClass().getResource("/Heart70x70.png");
        url[3] = getClass().getResource("/we70x70.png");
```

```
url[4] = getClass().getResource("/bomb70x70.png");
url[5] = getClass().getResource("/apple_red.png");

for(int i = 0 ; i < 6; i++)
    icon[i] = new ImageIcon(url[i]);

cl[0] = Color.RED;
cl[1] = Color.BLUE;
cl[2] = Color.CYAN;
cl[3] = Color.ORANGE;
cl[4] = Color.WHITE;
cl[5] = Color.PINK;

for(int i=0; i < 6; i++)
    for(int j=0; j < 10; j++ )
    {
        int d = rand();
        //bn[i][j] = new JButton(String.valueOf(d));
        bn[i][j] = new JButton();
        bn[i][j].setActionCommand(String.valueOf(d));
        bn[i][j].setBackground(cl[d]);
```

```
            bn[i][j].addActionListener(new Action());
            //bn[i][j].setFont(bn[i][j].getFont().deriveFont(48.0f));
            bn[i][j].setIcon(new ImageIcon(url[d]));
            bn[i][j].setOpaque(false);
            bn[i][j].setBorderPainted(false);



            jp.add(bn[i][j]);
        }





    jf.setVisible(true);

    check();
}

class Action implements ActionListener{
```

```
@Override
public void actionPerformed(ActionEvent e) {

    for(int i=0; i < 6; i++)
        for(int j=0; j < 10; j++ )
        {
            if(e.getSource() == bn[i][j])
            {
                if (x == -1 && y == -1) {
                    //Color tmp = Color.darkGray;

//bn[i][j].setBackground(bn[x][y].getBackground());
                    //bn[i][j].setBackground(tmp);
                    x = i;
                    y = j;
                    check();
                } else {
                    Color tmp = bn[i][j].getBackground();
                    String tmp1 = bn[i][j].getActionCommand();
                    String tmp12 = bn[x][y].getActionCommand();
```

```
                    ImageIcon tmp2 = new
ImageIcon(url[Integer.parseInt(tmp1)]);


                    if ((i == x + 1 && j == y) || (i == x - 1 && j ==
y) || (i == x  && j == y - 1) || (i == x && j == y + 1)) {

bn[i][j].setBackground(bn[x][y].getBackground());
                        bn[x][y].setBackground(tmp);



bn[i][j].setActionCommand(bn[x][y].getActionCommand());
                        bn[x][y].setActionCommand(tmp1);


                        bn[i][j].setIcon(new
ImageIcon(url[Integer.parseInt(tmp12)]));
                        bn[x][y].setIcon(tmp2);


                        //x=i;y=j;
                        x=-1; y=-1;


                        check();
                        moves--;
```

```
                    if(moves == 0)
                    {
                        int check =
JOptionPane.showConfirmDialog(null, "Your Score is:"+ score
+  " \nDo you want to Play again?");


                        if(check == JOptionPane.YES_OPTION)
                        {
                            new Candy();
                            jf.dispose();
                        }
                        else if(check ==
JOptionPane.NO_OPTION)
                            System.exit(0);
                    }
                }
                else
                {

//bn[x][y].setBackground(Color.LIGHT_GRAY);
                    //bn[i][j].setBackground(Color.darkGray);
```

```
                    x = i;
                    y = j;
                }
            }

            jb.setText("   Score: "+score+"
Moves: "+moves);

        }

    }

  }
}

public boolean check(){

    for(int i = 0; i < 6; i++)
    {
        for(int j = 0;j< 8; j++)
```

```
        {
            if(bn[i][j].getActionCommand().matches(bn[i][j +
1].getActionCommand()) &&
bn[i][j].getActionCommand().matches(bn[i][j +
2].getActionCommand())){


                if(j+ 3 < 10 &&
(bn[i][j].getActionCommand().matches(bn[i][j +
3].getActionCommand())))
                {
                    int d;


                    if(j + 4 < 10 &&
(bn[i][j].getActionCommand().matches(bn[i][j +
4].getActionCommand())))
                    {
                        d = rand();

bn[i][j+4].setActionCommand(String.valueOf(d));
                        bn[i][j+4].setBackground(cl[d]);
                        bn[i][j+4].setIcon(icon[d]);


                        score +=10;
```

```
                }

                    d = rand();

bn[i][j+3].setActionCommand(String.valueOf(d));
                    bn[i][j+3].setBackground(cl[d]);
                    bn[i][j+3].setIcon(icon[d]);

                    score+=10;

            }

            int d = rand();
            bn[i][j].setActionCommand(String.valueOf(d));
            bn[i][j].setBackground(cl[d]);
            bn[i][j].setIcon(icon[d]);

            d = rand();
            bn[i][j+1].setActionCommand(String.valueOf(d));
            bn[i][j+1].setBackground(cl[d]);
            bn[i][j+1].setIcon(icon[d]);
```

```
                d = rand();
                bn[i][j+2].setActionCommand(String.valueOf(d));
                bn[i][j+2].setBackground(cl[d]);
                bn[i][j+2].setIcon(icon[d]);

                score += 15;

                check();

                jb.setText("   Score: "+score+"
Moves: "+moves);
            }
        }
    }

    for(int j = 0;  j< 10; j++)
    {
        for(int i = 0;i< 4; i++)
        {
```

```
            if(bn[i][j].getActionCommand().matches(bn[i +
1][j].getActionCommand()) &&
bn[i][j].getActionCommand().matches(bn[i +
2][j].getActionCommand())){


                int d;


                if(i+ 3 < 6 &&
(bn[i][j].getActionCommand().matches(bn[i+3][j].getActionCo
mmand())))
                {


                    if(i+ 4 < 6 &&
(bn[i][j].getActionCommand().matches(bn[i+4][j].getActionCo
mmand())))
                    {
                        d = rand();

bn[i+4][j].setActionCommand(String.valueOf(d));
                        bn[i+4][j].setBackground(cl[d]);
                        bn[i+4][j].setIcon(icon[d]);
                        score+=10;
                    }
```

```
                d = rand();

bn[i+3][j].setActionCommand(String.valueOf(d));
                bn[i+3][j].setBackground(cl[d]);
                bn[i+3][j].setIcon(icon[d]);
                score +=10;

            }


            d = rand();
            bn[i][j].setActionCommand(String.valueOf(d));
            bn[i][j].setBackground(cl[d]);
            bn[i][j].setIcon(icon[d]);

            d = rand();
            bn[i+1][j].setActionCommand(String.valueOf(d));
            bn[i+1][j].setBackground(cl[d]);
            bn[i+1][j].setIcon(icon[d]);
```

```
                d = rand();
                bn[i+2][j].setActionCommand(String.valueOf(d));
                bn[i+2][j].setBackground(cl[d]);
                bn[i+2][j].setIcon(icon[d]);

                score += 15;

                check();

                jb.setText("   Score: "+score+"
Moves: "+moves);

            }
        }
    }




    return true;
  }
```

```
    public void down(int i, int j, int n)
    {
        for(int i1 = i+3; i1 > 0; i1-- )
        {
            String tmp12 = bn[i1-1][j].getActionCommand();
            bn[i1][j].setActionCommand(bn[i1-
1][j].getActionCommand());
            bn[i1][j].setIcon(new
ImageIcon(url[Integer.parseInt(tmp12)]));
        }
    }

    public void sleep(int n)
    {
        try {
            Thread.sleep(n);                //1000 milliseconds is one
second.
        } catch(InterruptedException ex) {
            Thread.currentThread().interrupt();
        }
    }
```

```
public int rand()
{
    Random r = new Random();
    return r.nextInt(6);

}

public static void main(String [] arg)
{
    new Candy();
}
                                        }
```

OBJECTS USED IN THE GAME AS CANDYS IN THE PNG FARMATE

Heart70x70.png

Chocolate70x70

Candy70x70

apple_red

bomb70x70

we70x70

GUI

```
<?xml version="1.0" encoding="UTF-8"?>
<module type="JAVA_MODULE" version="4">
  <component name="NewModuleRootManager" inherit-compiler-output="true">
    <exclude-output />
    <content url="file://$MODULE_DIR$">
      <sourceFolder url="file://$MODULE_DIR$" isTestSource="false" />
    </content>
    <orderEntry type="inheritedJdk" />
    <orderEntry type="sourceFolder" forTests="false" />
  </component>
</module>
```

CONCLUSION

The provided Java code lays the groundwork for a basic Candy Crush game using the Swing GUI framework. While functional, there is significant potential for future enhancements, including visual improvements, gameplay features such as power-ups and multiplayer modes, and optimizations to the user interface and game logic. The code offers a starting point for further development, and with additional refinements, it could evolve into a more polished and engaging gaming experience, aligning with contemporary standards and player expectations for interactive and entertaining applications.

FUTURE SCOPE

1. **User Interface Enhancement:**
    - Improve the visual design and layout for a more appealing user interface.
    - Add animations for candy swapping and matching to enhance the gaming experience.
2. **Level Design:**
    - Implement a level system with progressively increasing difficulty.
    - Introduce different challenges and goals for each level.
3. **Power-ups and Special Candies:**
    - Introduce special candies or power-ups with unique abilities.
    - Implement special effects or animations for power-up activations.
4. **Sound Effects and Music:**
    - Incorporate sound effects for candy matching, swapping, and other in-game events.
    - Add background music to enhance the overall gaming atmosphere.
5. **High Score Tracking:**
    - Implement a high score system to track and display the highest scores achieved by players.
    - Store high scores locally or remotely for competition among players.

6. **Multiplayer Mode:**
    - Implement a multiplayer mode, allowing players to compete or collaborate in real-time.
    - Introduce social features for sharing scores or challenging friends.
7. **Responsive Design:**
    - Ensure the game is responsive to different screen sizes and orientations.
    - Optimize the user interface for both desktop and mobile devices.
8. **Game Settings and Customization:**
    - Provide options for players to customize game settings such as difficulty level, grid size, or color themes.
9. **Refined Game Logic:**
    - Address any incomplete or unused code sections for a more comprehensive and stable game logic.
    - Optimize algorithms for candy matching and grid updates.
10. **Cross-Platform Compatibility:**
    - Explore the possibility of making the game cross-platform compatible.
    - Consider packaging the game as a standalone application or exploring web-based deployment.
11. **Localization:**
    - Implement localization support to make the game accessible to players in different languages and regions.

12. **Continuous Testing and Debugging:**

- Perform thorough testing to identify and fix any bugs or issues.
- Incorporate user feedback to enhance the overall gaming experience.

By addressing these areas of future scope, developers can take the initial Candy Crush game implementation to the next level, providing a more engaging and feature-rich gaming experience for players.

REFERENCES

1. Cay. S. Horstmann, Gary Cornell, "Core Java-JAVA Fundamentals", Prentice Hall, 10th ed., 2016.

2. Paul Deitel, Harvey Deitel, "Java SE 8 for programmers", 3rd Edition, Pearson, 2015.3. SCJP Sun Certified Programmer for Java 6 Study Guide. 6th edition, McGrawHill.

3. Herbert Schildt, "Introducing JavaFX 8 Programming", 1st Edition, McGraw Hill Education, New Delhi, 2015 for Unit V.

4. Introduction to Graphical User Interfaces with Java Swing by Paul Fischer (Author) 2005 1Edition

5. Learn Java™ GUI Applications A JFC Swing NetBeans Tutorial

8th Edition (2015) Philip Conrod & Lou Tylee.